

Smart Grid Management System - Frontend Functional Requirements

Frontend Design Specification

December 28, 2025

Context (Functional Requirements)

You are designing a SaaS-ready web application for the **Smart Grid Management System** targeting utility engineers, analysts, and energy managers in Cameroon. The app will allow users to register, manage workspaces, configure grids, define and run simulation scenarios, and view analytics on power distribution, failures, and optimization metrics.

Stack and Technical Constraints

- Next.js with JavaScript
- TailwindCSS for styling
- Lucide-React for icons
- Light/Dark theme support
- English/French language switch context
- State management using Zustand or Redux

Frontend Modules and Microservice Integration

1. Authentication & User Service

Purpose: Handle all user-related functionality.

Responsibilities:

- User registration, login/logout
- JWT/token-based authentication
- Role management (admin, regular user)
- Link users to their workspaces and scenarios

Persistence: Users table in DB

Reason for separation: Core SaaS requirement, may be implemented in a different language (Node.js, Python, etc.)

2. Workspace & Grid Configuration Service

Purpose: Manage all user-created workspaces and grid configurations.

Responsibilities:

- Create, read, update, delete (CRUD) workspaces
- Store grid topology (nodes, edges, components)
- Associate scenarios to workspaces
- Versioning or history (optional)

Persistence: Workspaces, Grid Configurations, Scenario mappings in DB

Reason for separation: Keeps persistent data isolated from simulation logic; can be used by multiple frontends or CLI independently

3. Scenario Management Service

Purpose: Manage simulation scenarios independently of grid definitions.

Responsibilities:

- Define, read, update, delete scenarios
- Predefined scenarios (line failure, plant outage, islanding)
- User-specific or shared scenarios

Persistence: Scenarios table in DB

Reason for separation: Modularizes scenario logic; allows reuse across multiple workspaces

4. Simulation & Optimization Engine

Purpose: Stateless core engine that runs simulations and optimization algorithms.

Responsibilities:

- Accept workspace + scenario input
- Run simulation algorithms (graph traversal, priority queues, union-find, segment trees)
- Return results (served loads, power distribution, metrics)
- Optionally store results for historical analysis

Statelessness: Each simulation is independent, no session memory

Reason for separation: Heavy computation, language flexibility (Python preferred), can scale independently

5. Analysis & Metrics Service

Purpose: Post-process simulation results and generate metrics/reports.

Responsibilities:

- Compute KPIs: total power generated vs. demand, served/unserved loads, system cost
- Generate analytics and visual summaries for web dashboard
- Optional export (CSV, JSON)

Reason for separation: Keeps computation of metrics separate; allows specialized libraries (Pandas, NumPy)